Introducing Apache Arrow: Columnar In-Memory Analytics

dremio.com/press-releases/introducing-apache-arrow-columnar-in-memory-analytics

Apache Arrow establishes a de-facto standard for columnar in-memory analytics which will redefine the performance and interoperability of most Big Data technologies. The lead developers of 13 major open source Big Data projects have joined forces to create Arrow, and additional companies and projects are expected to adopt and leverage the technology in the coming months. Within the next few years, I expect the vast majority of all new data in the world to move through Arrow's columnar in-memory layer.

Columnar in-memory complex analytics

Arrow grew out of three prevailing trends and business requirements:

- **Columnar**: Big Data has gone columnar. Led by the creation and adoption of Apache Parquet and other columnar data storage technologies, the industry has experienced rapid adoption of columnar storage for analytical workloads. These formats reduce the footprint and increase the performance of such workloads.
- **In-memory**: In-memory systems like SAP HANA and Spark accelerate analytical workloads by holding data in memory. In short, people are no longer willing to wait for non-in-memory systems.
- **Complex data and dynamic schemas**: Real-world objects are easier to represent as hierarchical and nested data structures. This has lead to the rise of formats/technologies such as JSON and document databases. Analytical systems must treat this type of data as first class.

There are many systems that support only one of the three requirements, and there are a few that support two of the three requirements. However, no existing system supports all three. The Big Data community has now come together to change this reality by creating the Apache Arrow project.

Arrow combines the benefits of columnar data structures with in-memory computing. It provides the performance benefits of these modern techniques while also providing the flexibility of complex data and dynamic schemas. And it does all of this in an open source and standardized way.

Arrow is a combination of data structures, algorithms and cross-language bindings

Arrow consists of a number of connected technologies designed to be integrated into storage and execution engines. The key components of Arrow include:

1. A set of defined data types, including both SQL and JSON types, such as Int, BigInt, Decimal, VarChar, Map, Struct and Array
2. Canonical columnar in-memory representations of data to support an arbitrarily complex record structure built on top of the defined data types
3. Common Arrow-aware companion data-structures including pick-lists, hash tables and queues
4. IPC through shared memory, TCP/IP and RDMA
5. Libraries for reading and writing columnar data in multiple languages including Java, C, C++ and Python
6. Pipelined and SIMD algorithms for various operations including bitmap selection, hashing, filtering, bucketing, sorting and matching
7. Columnar in-memory compression techniques to increase memory efficiency
8. Tools for short-term persistence to non-volatile memory, SSD or HDD

Arrow is not a storage or execution engine

Note that Arrow itself is not a storage or execution engine. It is designed to serve as a shared foundation for the following types of systems:

- SQL execution engines (such as Drill and Impala)
- Data analysis systems (such as Pandas and Spark)
- Streaming and queueing systems (such as Herron, Kafka and Storm)
- Storage systems (such as Parquet, Kudu, Cassandra and HBase)

As such, Arrow doesn’t compete with any of these projects. Its core goal is to work within each of these projects to provide faster performance and better interoperability. In fact, Arrow is being built by the lead developers of these projects.

Performance

The faster a user can get to the answer, the faster they can ask other questions. Performance begets more analysis, more use cases and further innovation. As CPUs become faster and more sophisticated, one of the key challenges is making sure processing technology leverages CPUs efficiently.

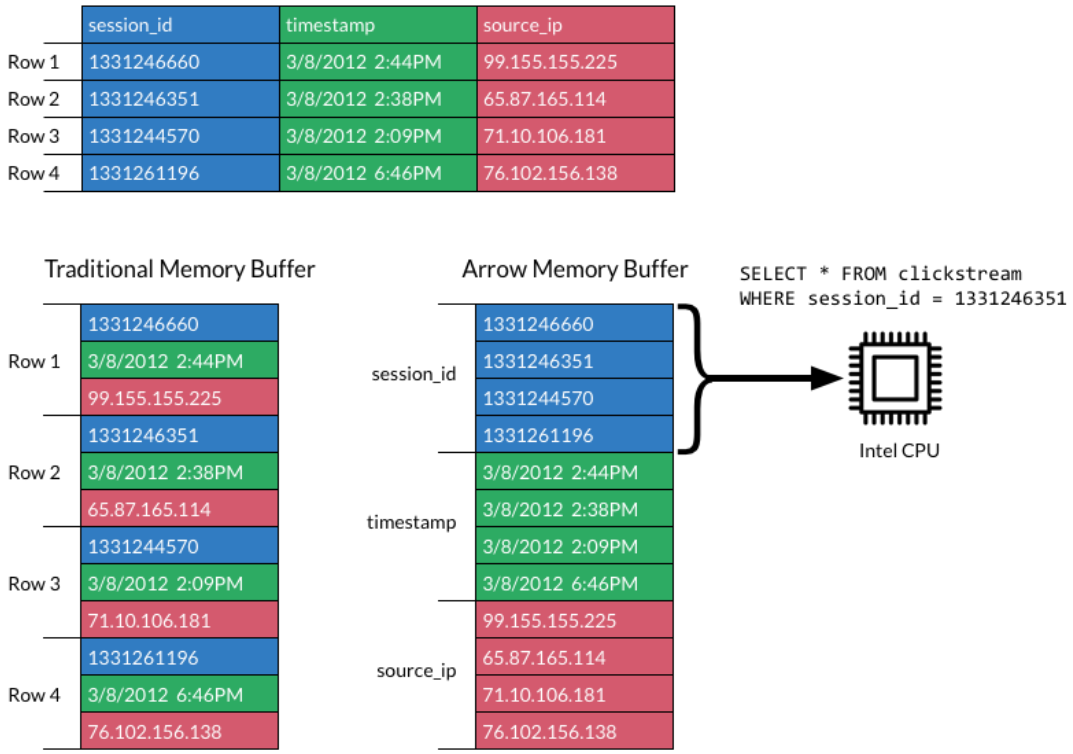
Columnar in-memory analytics is first and foremost about ensuring optimal use of modern CPUs. Arrow is specifically designed to maximize cache locality, pipelining and SIMD instructions.

Arrow memory buffers are compact representations of data designed for modern CPUs. The structures are defined linearly, matching typical read patterns. That means that data of similar type is collocated in memory. This makes cache prefetching more effective, minimizing CPU stalls resulting from cache misses and main memory accesses. These CPU-efficient data structures and access patterns extend to both traditional flat relational structures and modern complex data structures.

From there, execution patterns are designed to take advantage of the superscalar and pipelined nature of modern processors. This is done by minimizing in-loop instruction count and loop complexity. These tight loops lead to better performance and less branch-prediction failures.

Finally, the data structures themselves are designed for modern superword and SIMD instructions. Single Instruction Multiple Data (SIMD) instructions allow execution algorithms to operate more efficiently by executing multiple operations in a single clock cycle. In some cases, when using AVX instructions, these optimizations can increase performance by two orders of magnitude.

Cache locality, pipelining and superword operations frequently provide 10-100x faster execution performance. Since many analytical workloads are CPU bound, these benefits translate into dramatic end-user performance gains.These gains result in faster answers and higher levels of user concurrency.



Memory efficiency

In-memory performance is great, but memory can be scarce. Arrow is designed to work even if the data doesn't fit entirely in memory! The core data structure includes vectors of data and collections of these vectors (also called record batches). Record batches are typically 64KB-1MB, depending on the workload, and are always bounded at 2^16 records. This not only improves cache locality, but also makes in-memory computing possible even in low-memory situations. Arrow uses micro-pointers to minimize memory overhead and retrieval costs where inter-data-structure pointers exist.

With many Big Data clusters reaching 100s to 1000s of servers, systems must be able to take advantage of the aggregate memory of a cluster. Arrow is designed to minimize the cost of moving data on the network. It utilizes scatter/gather reads and writes and features a zero-serialization/deserialization design, allowing low-cost data movement between nodes. Arrow also works directly with RDMA-capable interconnects to provide a singular memory grid for larger in-memory workloads.

Zero-overhead data sharing

Arrow helps both execution engines and storage systems. It increases the internal performance of execution engines, and also provides a canonical way to communicate analytical data efficiently across systems.

Today there is no standard way for systems to transfer data. As a result, all communication involves serializing, copying an deserializing data. The serialization often takes place in the RPC layer, but there may also be implicit serialization imposed by the use of a shared set of record- and value-level APIs. In such cases, when working with multiple systems—such as Apache Drill reading from Apache Kudu—substantial resources and time are spent converting from one system's internal representation to the other system's internal representation.

As Arrow is adopted as the internal representation in each system, systems have the same internal representation of data. This means that one system can hand data directly to the other system for consumption. And when these systems are collocated on the same node, the copy described above can also be avoided through the use of shared memory. This means that in many cases, moving data between two systems will have zero overhead.

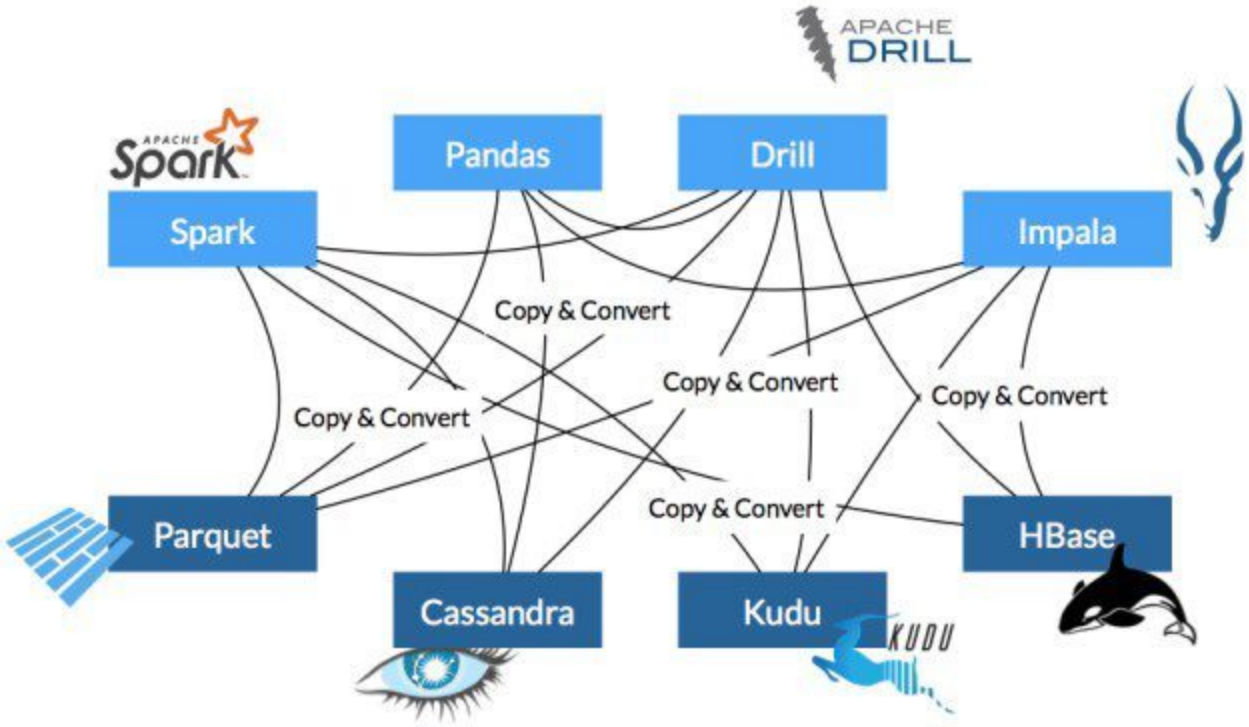


Figure 1: Before Apache Arrow

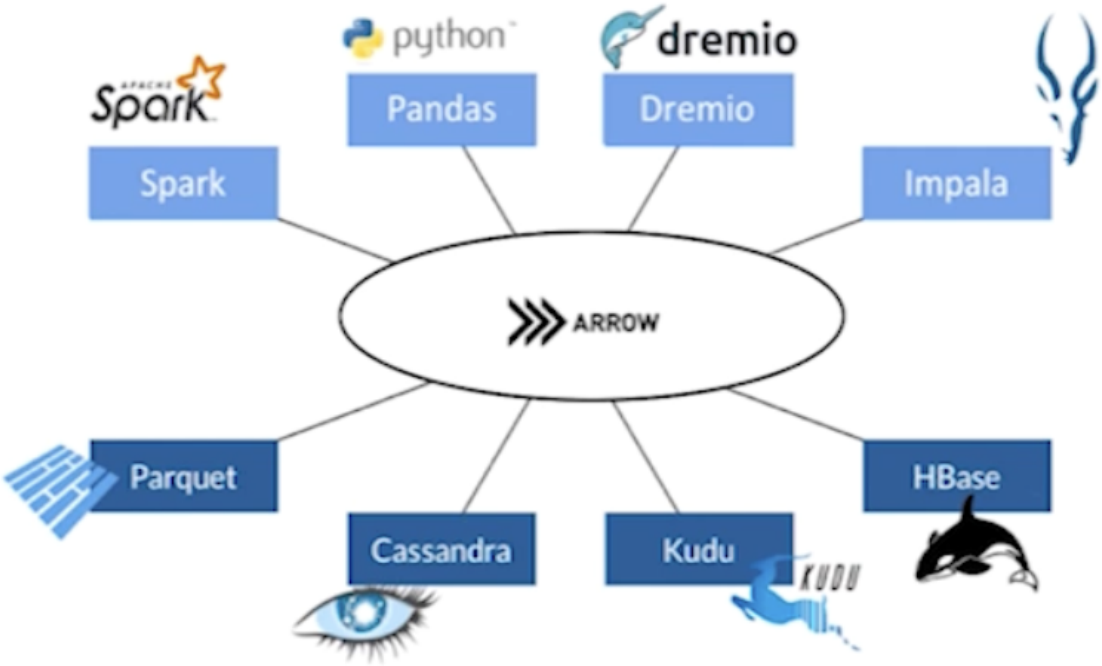


Figure 2: With Apache Arrow

First-class support for popular programming languages

One of the key outcomes users will see, beyond performance and interoperability, is a level-playing field among different programming languages. Traditional data sharing is based on IPC and API-level integrations. While this is often simple, it results in poor performance if the user's language is different from the underlying system's language. Depending on the language and the set of algorithms implemented, the required transformation often represent the majority of the processing time consumed.

This all changes with Arrow. By providing a canonical columnar in-memory representation of data, each programming language can interact directly with the raw data. C, C++, Java and Python Arrow libraries are underway and we will see integrations for R, Julia and JavaScript soon.

Open source based on a shared and acute need

Leading execution engines are continuously seeking opportunities to improve performance. Developers in the Drill, Impala, and Spark communities have all separately recognized the need for a columnar in-memory approach to data.

- Drill developers have implemented a columnar in-memory analytics layer inside Drill known as ValueVectors. Drill provides an execution layer that performs SQL processing directly on columnar data without row materialization. The combination of optimizations for columnar storage and direct columnar execution significantly lowers memory footprints and provides faster execution of BI and SQL workloads.

- Impala developers also forecasted the need for a columnar in-memory approach in their 2015 CIDR paper: “We are also considering switching to a columnar canonical in-memory format for data that needs to be materialized during query processing, in order to take advantage of SIMD instructions” (http://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper28.pdf)
- Spark developers also recognized this need, speaking of similar goals when outlining their vision for the Spark Tungsten initiative: “We’ve found that a large fraction of the CPU time is spent waiting for data to be fetched from main memory. As part of Project Tungsten, we are designing cache-friendly algorithms and data structures so Spark applications will spend less time waiting to fetch data from memory and more time doing useful work” (https://databricks.com/blog/2015/04/28/project-tungsten-bringing-spark-closer-to-bare-metal.html)

The Big Data community has recognized an opportunity to develop a shared technology to address columnar in-memory analytics, and has joined forces to create Apache Arrow. The Apache Drill community is seeding the project with the Java library, based on Drill’s existing ValueVectors technology, and Wes McKinney, creator of Pandas and Ibis, is contributing the initial C/C++ library. Given the credentials of those involved as well as code provenance, the Apache Software Foundation decided to make Apache Arrow a Top-Level Project, highlighting the importance of the project and community behind it.

A diverse community of open source leaders

Many lead developers (PMC Chairs, PMC Members, Committers, etc.) of **14** major open source projects are involved in Apache Arrow:

- Calcite
- Cassandra
- Dremio
- Drill
- Hadoop
- HBase
- Ibis
- Impala
- Kudu
- Pandas
- Parquet
- Phoenix
- Spark
- Storm

What’s next?

The code for Apache Arrow is now available under the Apache 2.0 License for use in both open source and proprietary systems. Arrow implementations and bindings are available or underway for C, C++, Java and Python.

Dremio, Drill, Impala, Kudu, Ibis and Spark will become Arrow-enabled this year, and I anticipate that many other projects will embrace Arrow in the near future as well. Arrow community members (including myself) will be speaking at upcoming conferences including Strata San Jose, Strata London and numerous meetups.

Join the community today. Follow @ApacheArrow, connect on the dev@arrow.apache.org list or check out the code at https://github.com/apache/arrow.